

in this prompt i will be explain the whole workflow and features of the extension we have to make. so our extension will have two main features one project evolution graph and planning graph.

1 – Evolution graph will make a vertical tree like 3d graph which will only represent the workflow of the project mainly the business logic in detail covering all the features and connecting them to represent the user workflow and connect all the elements to their respected features. it will be detailed in terms of representing the business/project logic. evolution graph will show each and every element we have created for example “from login node goes a branch which shows email input field, then another node that shows the submit button “. and each element node will contain the location of the code related to its which will be classified and when the element is removed or have an error or changes it will show a drifted sign or error. The main idea is developers will be able to navigate the business workflow of the project

2 – Plan Map's primary visual interface for understanding how a piece of software works. It generates a Plan Graph: a spatial, zoomable map of the software, where the developer can see the whole system, zoom into any single feature, and view any part through different technical lenses. The Plan Graph exists so the developer understands and approves how the software will work — at the level of abstraction they care about — before any code is written.

Core principle: *Understand the software before seeing the code.* The graph shows business behaviour first; technical implementation is revealed progressively, on demand, through zoom and lenses.

The Two Zoom Levels

Level 1 — The Constellation (whole-project view)

The top-down, zoomed-out view of the entire project at once. What a node/cluster represents: a whole feature or workflow — e.g. *Browse Restaurants, Register / Login, Cart, Checkout, Order Tracking, Ratings*. Not in details like evolution graph which represent all the elements of each feature it will be a simple vertical workflow of the project

What you see: every feature of the project as a distinct, **color-coded** region. You are NOT looking at individual steps here — you are looking at the shape of the whole system, the way constellations group stars.

Why it exists: it answers "how does the graph survive a huge prompt?" — a prompt like "*make me a complete food ordering application*" has enormous blast radius and many features. Instead of cramming all of it into one flow, PlanMap lays the features out as separate clustered regions you can take in at a glance.

How clusters are connected: by **user journey** (arrows showing how a user moves feature-to-feature: browse → cart → checkout → tracking) — and, when the lens is switched, by **backend workflow** (how requests/data move between features at the system level)

Level 2 — The Feature Space (zoom-in view)

Click a single feature at the Constellation level and the canvas **zooms into that region** spatially, like flying down into one area of a map ("we zoom it into a space").

What a node represents: a **meaningful step in that feature's workflow** — not a whole feature anymore.

Example (Login feature, Business lens):

` land on domain → home page → click Register/Login → enter email → enter password → complete reCAPTCHA → press Enter → dashboard opens `

Each node is a real, understandable software event. This is the detailed, walkable flow of that one feature.

Navigation metaphor to keep consistent:

Zoom = moving between altitudes (Constellation ↔ Feature Space). It is *spatial* — you fly down into a region and back out.

3. The Lens System

Lenses are the second navigation axis, orthogonal to zoom. Where zoom changes your *altitude*, a lens changes what the nodes mean — without changing which feature or which altitude you're at.

Critical decision (locked): lenses apply at BOTH zoom levels — the Constellation and the Feature Space — not only inside a feature.

At the Constellation level

Business lens: features connected by user journey (how a person flows through the product).

Backend lens: the *same* features, re-connected by how requests and data actually move between them at the system level.

At the Feature Space level (inside one feature, e.g. Login)

The same feature's flow, seen three ways:

Business lens — user-facing steps:

` land → home → click Register → email → password → reCAPTCHA → submit → dashboard `

Backend lens — the same login as request execution through the code:

` API → route → middleware → controller → service → DB → response `

Security lens — the same login as trust boundaries:

` credentials received → input validation → password hash verification → JWT issued → authorization → access granted `

The key property: it is the *same* feature and the *same* underlying data every time. The lens only changes how the request-through-the-system is portrayed. Switching lenses does **not** move you to a different feature and does **not** change your zoom altitude.

Navigation metaphor to keep consistent:

Lens = staying at the same altitude but changing what the nodes *represent* (business steps vs. backend calls vs. security boundaries).

4. How a Large Prompt Flows Through the System

Worked example, end to end, for the hardest case:

Prompt: *"Make me a complete food ordering application."*

- **Triage** classifies this as very high blast radius → structured path → Plan Graph generation.
- **Constellation generated.** PlanMap produces the whole-project map: every feature as a color-coded cluster (Browse Restaurants, Register/Login, Cart, Checkout, Payments, Order Tracking, Ratings...), connected by user journey under the default Business lens.
- **Developer surveys the whole system** at a glance — sees every workflow and feature the app will contain, before a line of code exists.
- **Developer clicks one feature** (e.g. *Register/Login*). The canvas **zooms into that Feature Space**, revealing the detailed business-logic flow of just that feature.
- **Developer switches lens** inside the feature to inspect it as backend execution or as security boundaries — same feature, different portrayal.

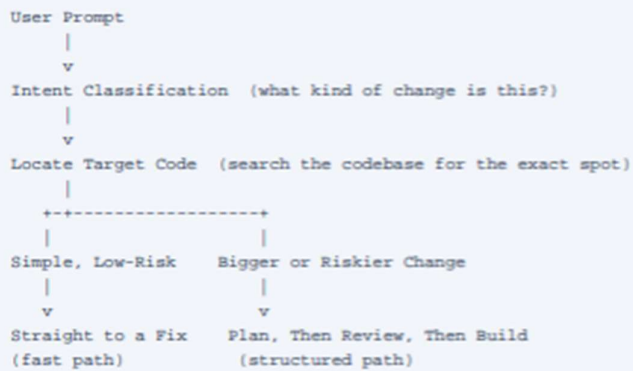
- **Developer reviews/edits/approves** at whichever level is relevant. Approved work proceeds to the coding agent (per the main pipeline).

This is the answer to the original design worry ("a huge prompt has too many features and too much business logic for one graph"): **the graph never tries to show everything at one altitude.** Breadth lives at the Constellation level; depth lives in each Feature Space; the developer moves between them by zooming.

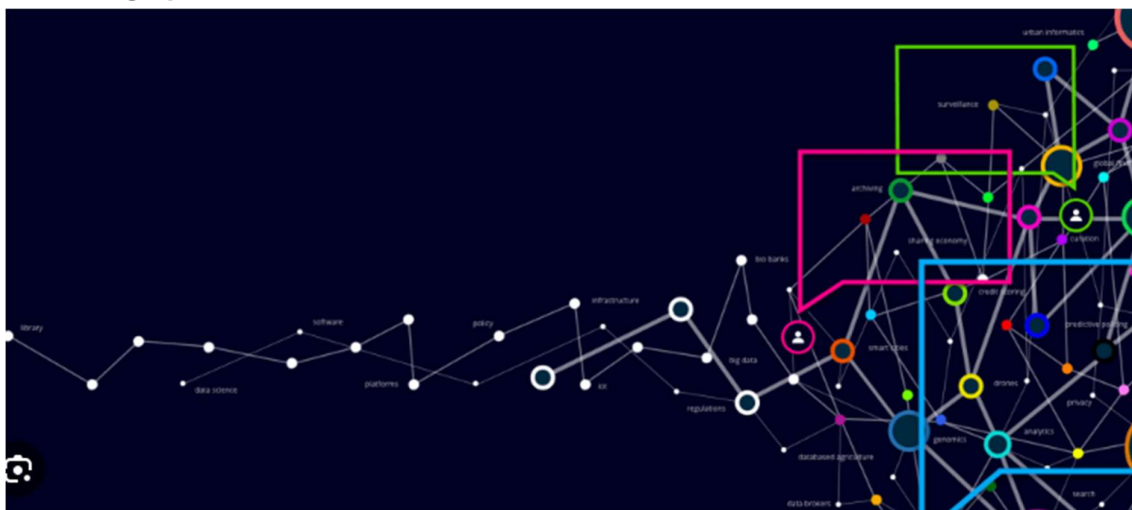
•

6 How a Request Is Handled, Step by Step

Every request goes through the same first checkpoint:



How the graph should look -





In the images u see I don't want it to be exactly like that I want a vertically growing graph with connection shown properly it should be a 3d graph and when we hover on a node the info block should be seen .

How the extension will work,

It will start then enter your prompt file etc. then it will classify the prompt it is a feature add on or creating the whole project so it will work and make the plan graph accordingly after reading the existing code. if it is a simple fix it like change the colour or make the section bigger it will find the code which effect that element and make exact prompt so the coding agent just have to execute it and not think and work on it. once the plan graph is made the developer use lenses and can make change to each and every node. he can manually add a node also. once everything is approved and verified the details prompt will be generated that will tell exactly what to do to the coding agent. and the project evolution graph will be made after reading the code ai have made.